資料分析與機器學習 (Data Analysis and Machine Learning)

深度神經網路能降低表達複雜函式的參數需求量，但需要特別的演算法與技術來訓練 (深度學習)

Lecture 7 –

Multilayer Neural Networks & PyTorch

教師：陳慶永

Outline

- Multilayer Artificial Neural Network (NN)
  - Network architecture
  - Activating NNs via forward propagation
  - Training NNs via backpropagation
  - Activation functions for multilayer NNs
- Implementing a Multilayer Perceptron (MLP) from Scratch
- PyTorch
- Building and Training an MLP in PyTorch

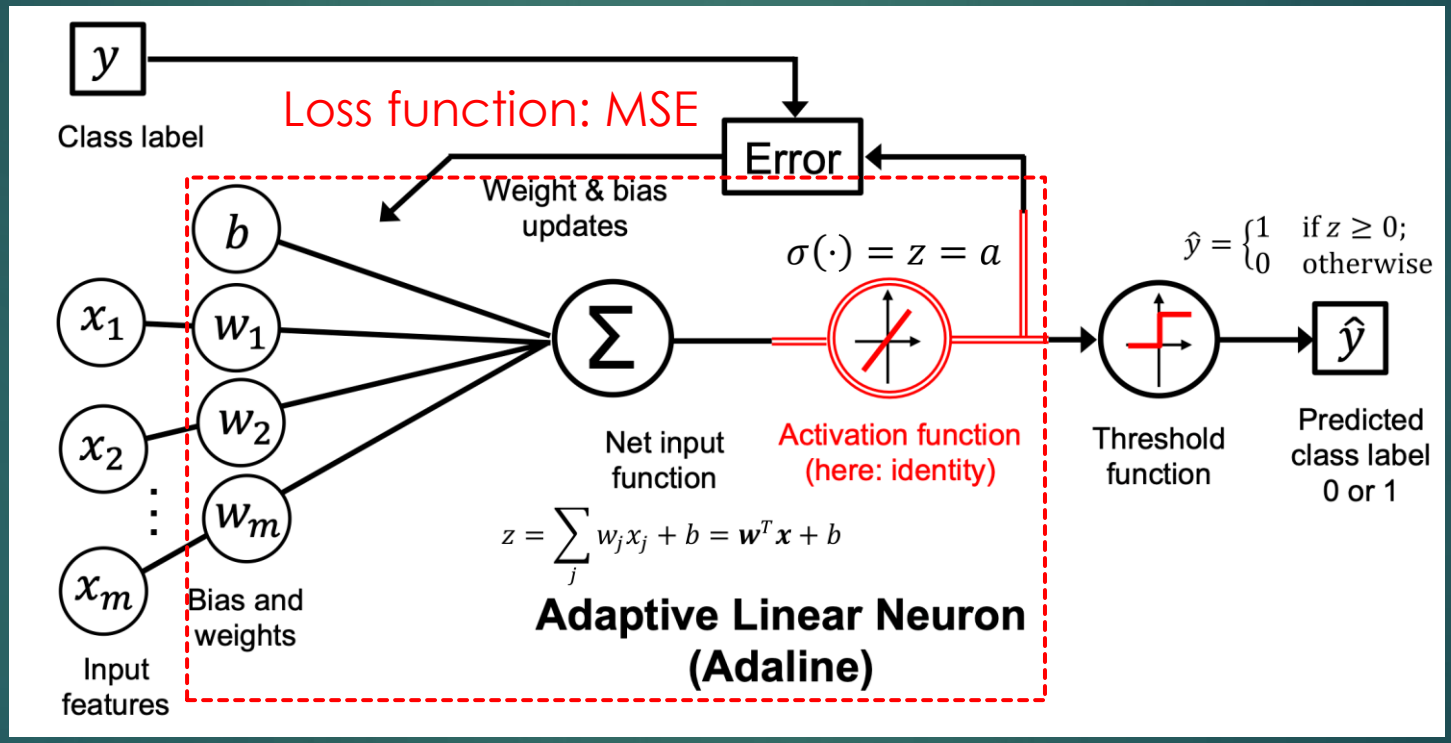
Multilayer Artificial Neural Network (NN)

Deep Learning

- A *subfield* of machine learning
  - Training artificial neural networks (NNs) with many layers efficiently
- Our study strategy
  - Learn the basic concepts of multilayer NNs
  - Introduce a deep learning library (PyTorch)
  - Building and training a deep NN (DNN) in PyTorch

Single-layer NN

- Adaline (review Lecture 3)



It is called a single-layer network because of its single link (neuron) between the input and output.

Multilayer NN

- Multilayer Perceptron (MLP)
  - *Fully connected* (全連接) network
  - Example: 2-layer MLP
    - Hidden layer (d neurons) fully connected to m input features

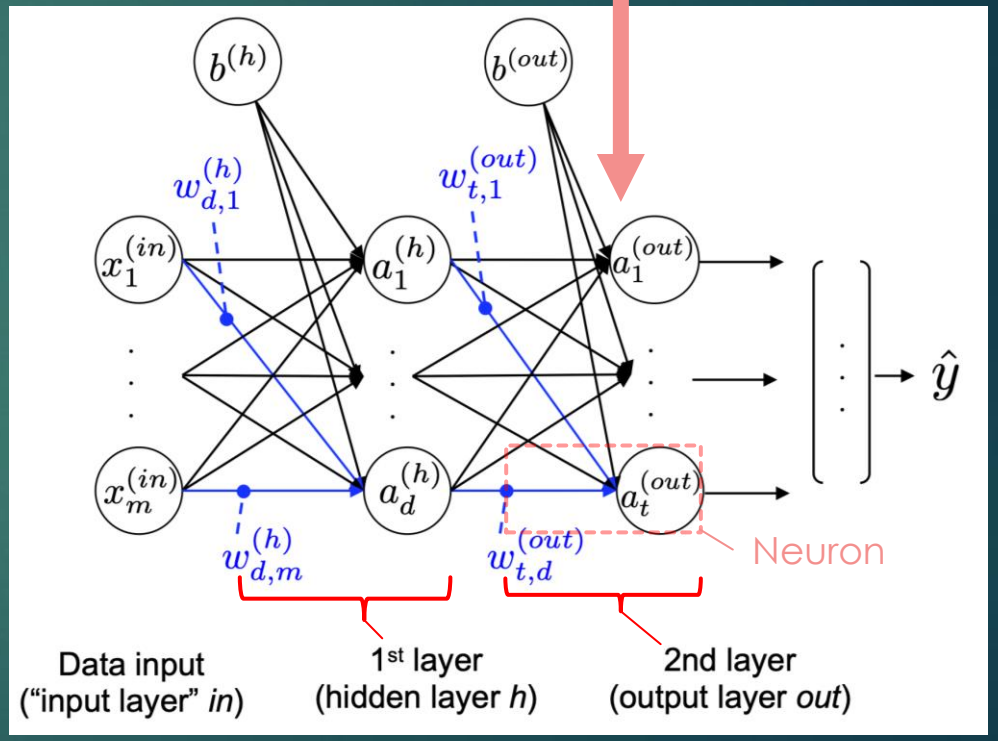
$$z_1^{(h)} = x_1^{(in)} w_{1,1}^{(h)} + x_2^{(in)} w_{1,2}^{(h)} + \cdots + x_m^{(in)} w_{1,m}^{(h)} + b_1^{(h)}$$

$$a_1^{(h)} = \sigma\left(z_1^{(h)}\right)$$

Activation function

    - Output layer (t neurons) fully connected to hidden layer

A column of neurons forms a **layer**



d 個神經元 (節點 / 單元)

t 個神經元 (節點 / 單元)

Multilayer NN

- *Nonlinear* activation functions (e.g., sigmoid) are used to solve complex problems
  - The term multilayer *perceptron* may sound a little bit confusing since the neurons are typically sigmoid units, *not* perceptrons

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

  - Think of the neurons in the MLP as logistic regression units

Multilayer NN

- Universal approximation theorem states that a feedforward NN with a single hidden layer and a relatively large number of hidden units can approximate arbitrary continuous functions
  - Adding more hidden units corresponds to increasing the *width* of a layer
- Alternatively, we can add more hidden layers to create *deeper* network architectures
  - A network with more than one hidden layer is a deep NN (DNN) (深度神經網路)

Multilayer NN

- Numbers of layers and units in an NN are additional hyperparameters that we want to optimize via cross-validation
  - Advantage of making a network deeper rather than wider is that fewer parameters are required to achieve a comparable model capacity
  - However, vanishing gradient (梯度消失) problem makes model learning of a DNN more challenging
    - Loss gradients for updating parameters become increasingly small for deeper NN
    - Special algorithms have been developed to help train such DNN structures (known as deep learning (深度學習))

Activating NNs via Forward Propagation

- Hidden layer (m features, d units)
  - Weight matrix $(d \times m)$: $\boldsymbol{W}^{(h)}$
  - Bias vector $(1 \times d)$: $\boldsymbol{b}^{(h)}$
  - Feature vector $(1 \times m)$ of a single sample: $\boldsymbol{x}^{(in)}$
    - Net input vector $(1 \times d)$: $\boldsymbol{z}^{(h)} = \boldsymbol{x}^{(in)}\left(\boldsymbol{W}^{(h)}\right)^T + \boldsymbol{b}^{(h)}$
    - Activation vector $(1 \times d)$: $\boldsymbol{a}^{(h)} = \sigma\left(\boldsymbol{z}^{(h)}\right)$
  - Feature matrix $(n \times m)$ of n samples: $\boldsymbol{X}^{(in)}$
    - Net input matrix $(n \times d)$: $\boldsymbol{Z}^{(h)} = \boldsymbol{X}^{(in)}\left(\boldsymbol{W}^{(h)}\right)^T + \boldsymbol{B}^{(h)}$
    - Activation matrix $(n \times d)$: $\boldsymbol{A}^{(h)} = \sigma\left(\boldsymbol{Z}^{(h)}\right)$

大寫粗體表示矩陣
小寫粗體表示向量

Input of network

$$\begin{bmatrix} \boldsymbol{b}^{(h)} \\ \vdots \\ \boldsymbol{b}^{(h)} \end{bmatrix}_{n \times d}$$

Activating NNs via Forward Propagation

- Output layer (d hidden inputs, t units)
  - Weight matrix $(t \times d)$: $\boldsymbol{W}^{(out)}$
  - Bias vector $(1 \times t)$: $\boldsymbol{b}^{(out)}$
  - Activation matrix $(n \times d)$ of hidden layer: $\boldsymbol{A}^{(h)}$
    - Net input matrix $(n \times t)$: $\boldsymbol{Z}^{(out)} = \boldsymbol{A}^{(h)}\left(\boldsymbol{W}^{(out)}\right)^T + \boldsymbol{B}^{(out)}$
    - Activation matrix $(n \times t)$: $\boldsymbol{A}^{(out)} = \sigma\left(\boldsymbol{Z}^{(out)}\right)$

$$\begin{bmatrix} \boldsymbol{b}^{(out)} \\ \vdots \\ \boldsymbol{b}^{(out)} \end{bmatrix}_{n \times t}$$

Output of network

每一個樣本都有 t 個輸出 (機率)
可用於多分類問題

Training NNs via Backpropagation

- *Training* an MLP for multiclass classification
  - Need to choose a loss function $L(\boldsymbol{W}, \boldsymbol{b})$ to calculate the *error* between the output and the target
    - Example: MSE (just for the ease of illustration)
      - Average over t activation units
      - Average over n samples

$$L(\boldsymbol{W}, \boldsymbol{b}) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{t}\sum_{j=1}^{t}\left(y_j^{[i]} - a_j^{(out)[i]}\right)^2$$

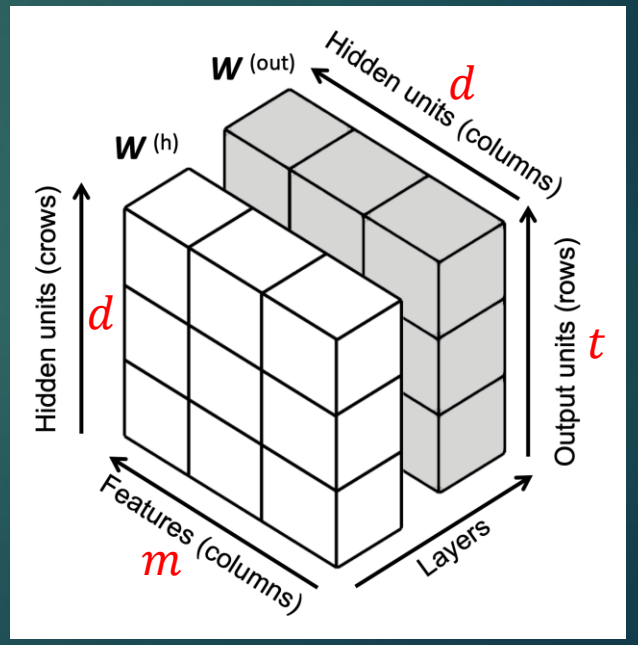
Superscript $[i]$ is the index of a particular *sample* in the training dataset.

Training NNs via Backpropagation

- $\boldsymbol{W}$ consists of multiple matrices ($\boldsymbol{W}^{(h)}$ and $\boldsymbol{W}^{(out)}$)
  - To minimize the loss function $L(\boldsymbol{W}, \boldsymbol{b})$, we need to calculate its partial derivative w.r.t *each weight* for *every layer* in the network

$$\frac{\partial}{\partial w_{j,k}^{(l)}} L(\boldsymbol{W}, \boldsymbol{b})$$

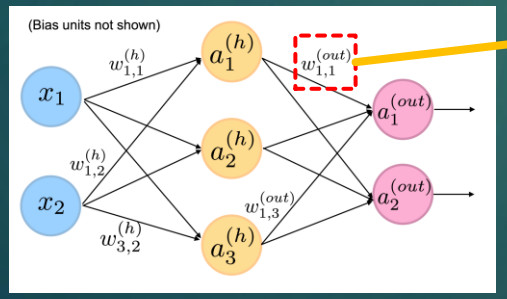
  - A popular solution
    - The backpropagation algorithm

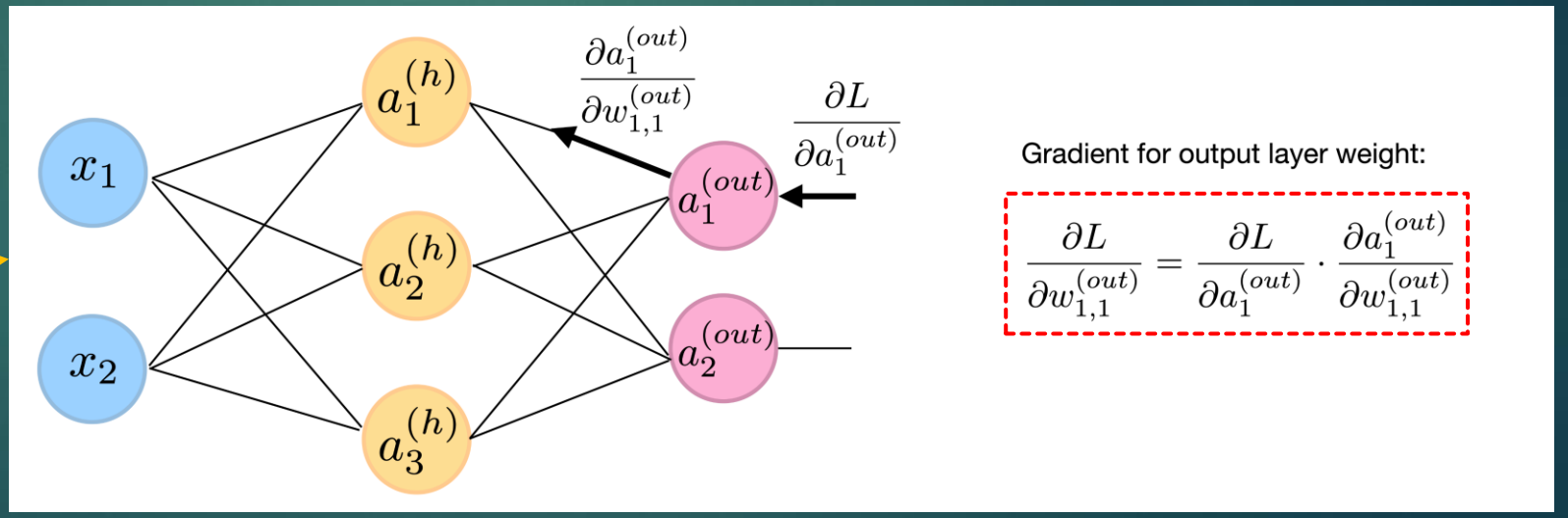


Visualization of the 3D tensor $\boldsymbol{W}$

Backpropagation Algorithm

- Backpropagation is simply a special case of *reverse-mode* automatic differentiation
  - Traverse the chain rule *from the output*
  - Example: Gradient for the output layer weight





Gradient for output layer weight:

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

Backpropagation Algorithm

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

$$\frac{\partial L}{\partial a_1^{(out)}} = \frac{\partial}{\partial a_1^{(out)}} \left(y_1 - a_1^{(out)}\right)^2 = 2\left(a_1^{(out)} - y\right)$$

$$\frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} = \frac{\partial}{\partial z_1^{(out)}} \frac{1}{1 + \bar{e}^{z_1^{(out)}}} = \quad \dots \quad = \left(\frac{1}{1 + \bar{e}^{z_1^{(out)}}}\right)\left(1 - \frac{1}{1 + \bar{e}^{z_1^{(out)}}}\right)$$

$$= a_1^{(out)}\left(1 - a_1^{(out)}\right)$$

$$\frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}} = \frac{\partial}{\partial w_{1,1}^{(out)}} a_1^{(h)} w_{1,1}^{(out)} + b_1^{(out)} = a_1^{(h)}$$

For simplicity, averaging over individual samples $\frac{1}{n}\sum_{i=1}^{n}$ is omitted

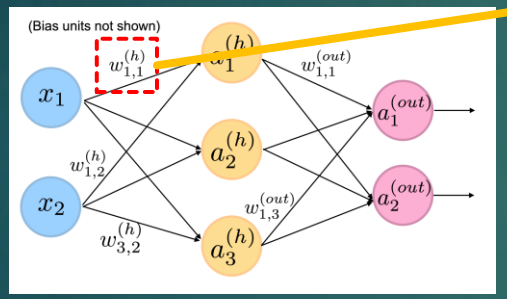
$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}} = 2\left(a_1^{(out)} - y\right) \cdot a_1^{(out)}\left(1 - a_1^{(out)}\right) \cdot a_1^{(h)}$$

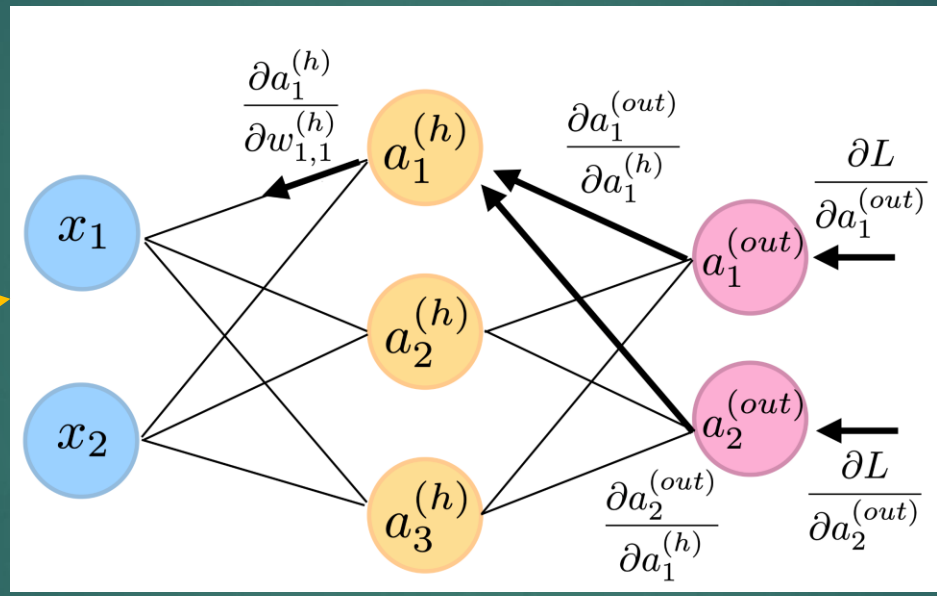
$$w_{1,1}^{(out)} := w_{1,1}^{(out)} - \eta \frac{\partial L}{\partial w_{1,1}^{(out)}}$$

Weight update by GD

Backpropagation Algorithm

- Example: Gradient for the hidden layer weight





Gradient for hidden layer weight:

$$\frac{\partial L}{\partial w^{(h)}_{1,1}} = \frac{\partial L}{\partial a^{(out)}_1} \cdot \frac{\partial a^{(out)}_1}{\partial a^{(h)}_1} \cdot \frac{\partial a^{(h)}_1}{\partial w^{(h)}_{1,1}} + \frac{\partial L}{\partial a^{(out)}_2} \cdot \frac{\partial a^{(out)}_2}{\partial a^{(h)}_1} \cdot \frac{\partial a^{(h)}_1}{\partial w^{(h)}_{1,1}}$$

Note: The weight $w^{(h)}_{1,1}$ is connected to both output nodes, we have to use the multi-variable chain rule to sum the two paths highlighted with bold arrows.

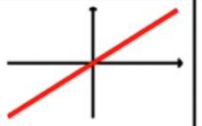
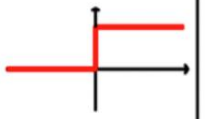
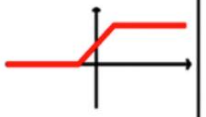
Activation Functions

- Technically, any *differentiable* function *can* be used as an activation function in multilayer NNs
  - In practice, it would not be very useful to use linear activation functions for both hidden and output layers
    - The *sum of linear functions* yields a linear function after all
  - We want to introduce *nonlinearity* in a typical artificial NN to be able to tackle complex problems

Common Nonlinear Activation Functions

- Logistic (sigmoid): $\sigma(z) = \frac{1}{1+e^{-z}}$, Range: $(0,1)$
  - Output layer for binary classification
- Softmax: $\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{M} e^{z_j}}$, Range: $(0,1)$
  - Output layer for multi-class classification
- Hyperbolic tangent (tanh): $\sigma(z) = \tanh z = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$, Range: $(-1,1)$
  - Hidden layers in RNNs
- Rectified Linear Unit (ReLU): $\sigma(z) = \max(0, z)$, Range: $[0, \infty)$
  - Hidden layers in CNNs and DNNs
- Leaky ReLU: $\sigma(z) = \max(\alpha z, z)$, Range: $(-\infty, \infty)$
  - Hidden layers when standard ReLU fails

Graphs of Activation Functions

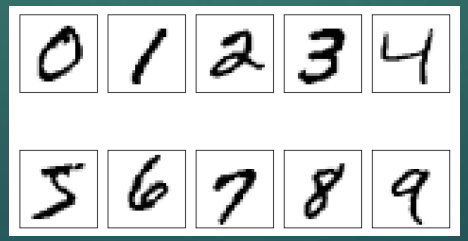
Activation function	Equation	Example	1D graph
Linear	$\sigma(z) = z$	Adaline, linear regression	
Unit step (Heaviside function)	$\sigma(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\sigma(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise linear	$\sigma(z) = \begin{cases} 0 & z \le -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \le z \le \frac{1}{2} \\ 1 & z \ge \frac{1}{2} \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\sigma(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, multilayer NN	
Hyperbolic tangent (tanh)	$\sigma(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\sigma(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

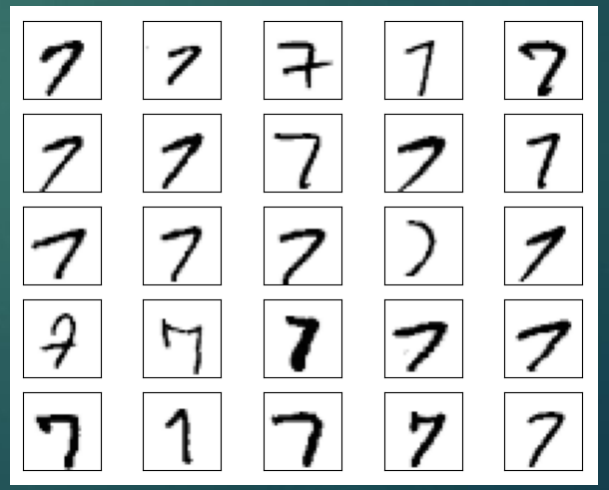
Implementing a Multilayer Perceptron (MLP) from Scratch

Classifying Handwritten Digits

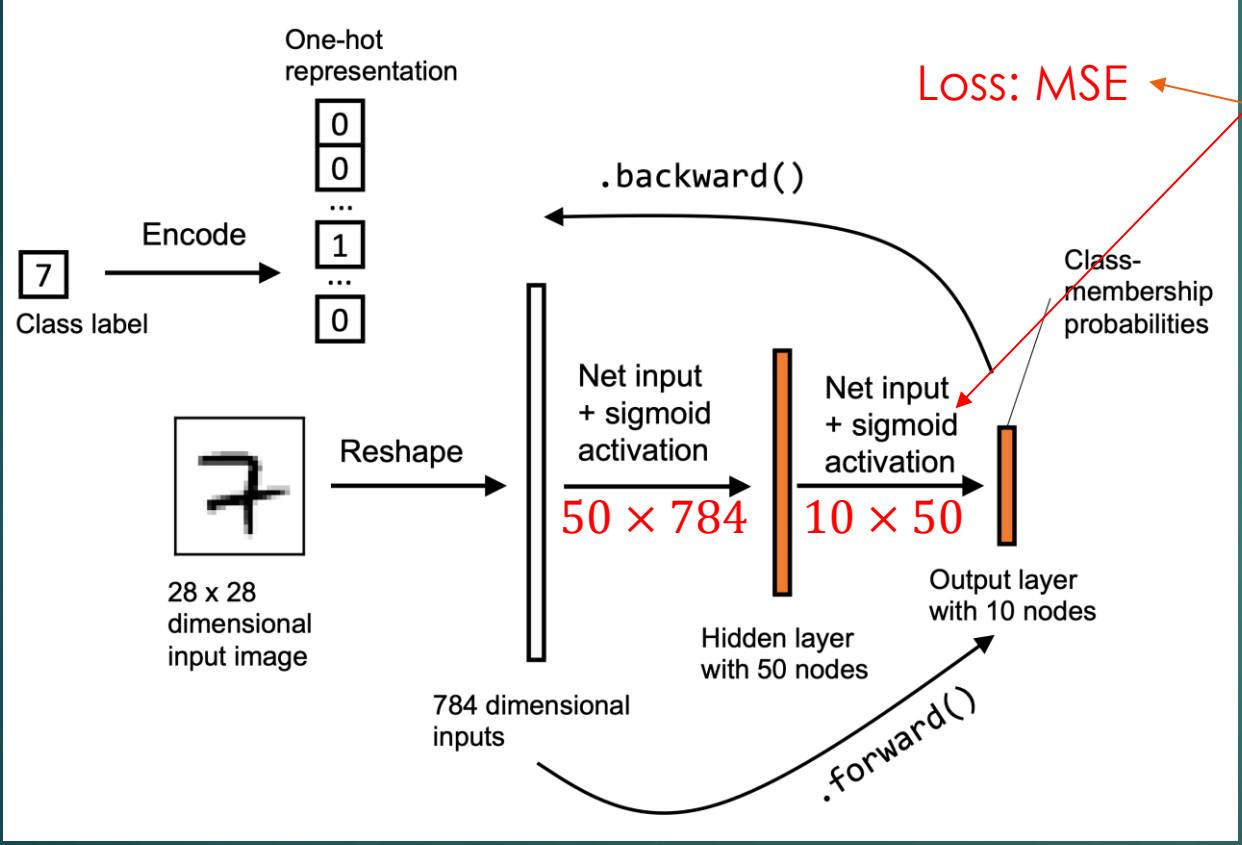
- Implement and train our first multilayer NN to classify handwritten digits from the popular MNIST dataset
  - 70,000 images of resolution 28×28 (grayscale)
  - Features: 784 pixels (integer $[0, 255]$)
  - Labels: (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) (10 classes)

```
# Load the MNIST dataset
X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
X = X.values
y = y.astype(int).values
```





Implementing an MLP



(對於多分類問題不是最佳的選擇，
單純只是為了與 pp. 6-15 的推導呼應)

```
class NeuralNetMLP:
    def __init__(self, num_features, num_hidden, num_classes, random_seed=123):···

    def forward(self, x):···

    def backward(self, x, a_h, a_out, y):       ···

model = NeuralNetMLP(num_features=28*28,
                     num_hidden=50,
                     num_classes=10)
```

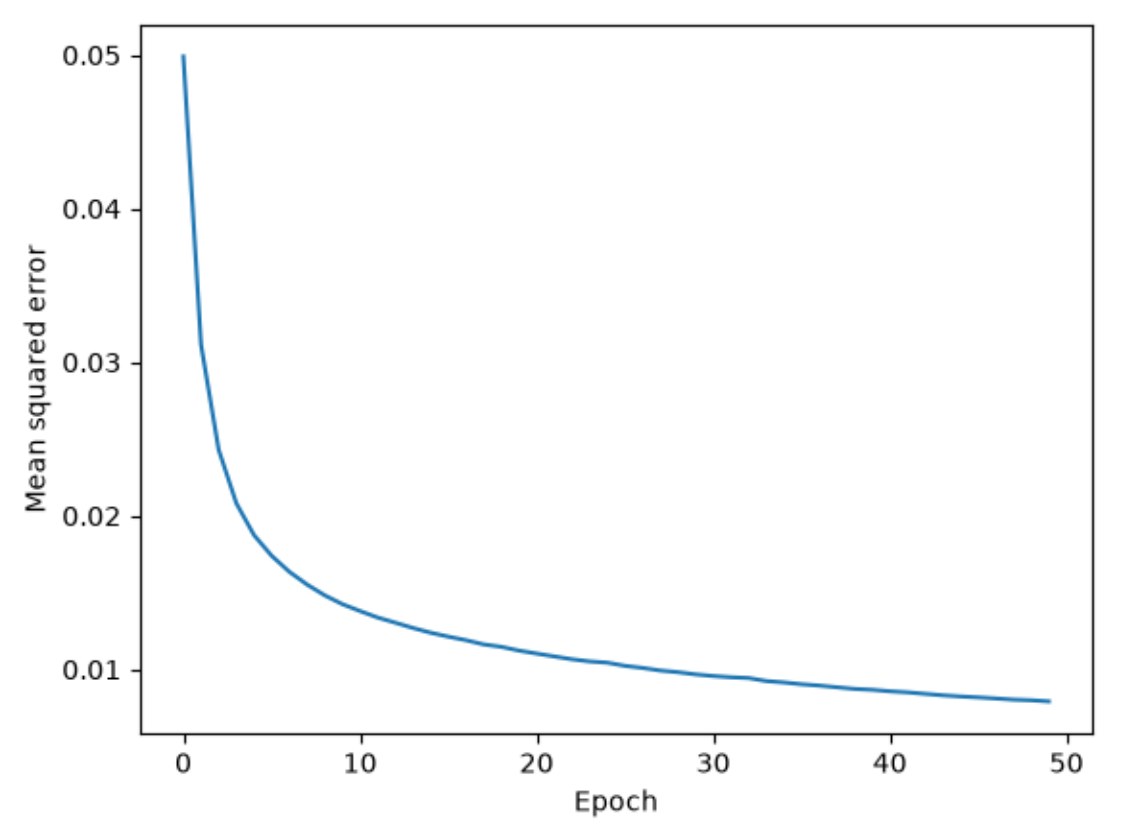
Training an MLP

- Sample code: 07_1.py
  - Load the MNIST dataset
    - Normalize pixels to $[-1, 1]$ range (usually works well in practice)
  - Split into training, validation $(5{,}000)$ and test set $(10{,}000)$
  - Implement the MLP
  - Define mini-batch data loader and training loop
    - Batch size: 64
  - Train the MLP
    - Number of epochs: 50
  - Evaluate the performance

```
Epoch: 001/050 | Train MSE: 0.05 | Train Acc: 76.15% | Valid Acc: 75.98%
Epoch: 002/050 | Train MSE: 0.03 | Train Acc: 85.45% | Valid Acc: 85.04%
Epoch: 003/050 | Train MSE: 0.02 | Train Acc: 87.82% | Valid Acc: 87.60%
Epoch: 004/050 | Train MSE: 0.02 | Train Acc: 89.36% | Valid Acc: 89.28%
Epoch: 005/050 | Train MSE: 0.02 | Train Acc: 90.21% | Valid Acc: 90.04%
Epoch: 006/050 | Train MSE: 0.02 | Train Acc: 90.67% | Valid Acc: 90.54%
Epoch: 007/050 | Train MSE: 0.02 | Train Acc: 91.12% | Valid Acc: 90.82%
Epoch: 008/050 | Train MSE: 0.02 | Train Acc: 91.43% | Valid Acc: 91.26%
Epoch: 009/050 | Train MSE: 0.01 | Train Acc: 91.84% | Valid Acc: 91.50%
Epoch: 010/050 | Train MSE: 0.01 | Train Acc: 92.04% | Valid Acc: 91.84%
Epoch: 011/050 | Train MSE: 0.01 | Train Acc: 92.30% | Valid Acc: 92.08%
Epoch: 012/050 | Train MSE: 0.01 | Train Acc: 92.51% | Valid Acc: 92.24%
Epoch: 013/050 | Train MSE: 0.01 | Train Acc: 92.65% | Valid Acc: 92.30%
Epoch: 014/050 | Train MSE: 0.01 | Train Acc: 92.80% | Valid Acc: 92.60%
Epoch: 015/050 | Train MSE: 0.01 | Train Acc: 93.04% | Valid Acc: 92.78%
Epoch: 016/050 | Train MSE: 0.01 | Train Acc: 93.14% | Valid Acc: 92.68%
```

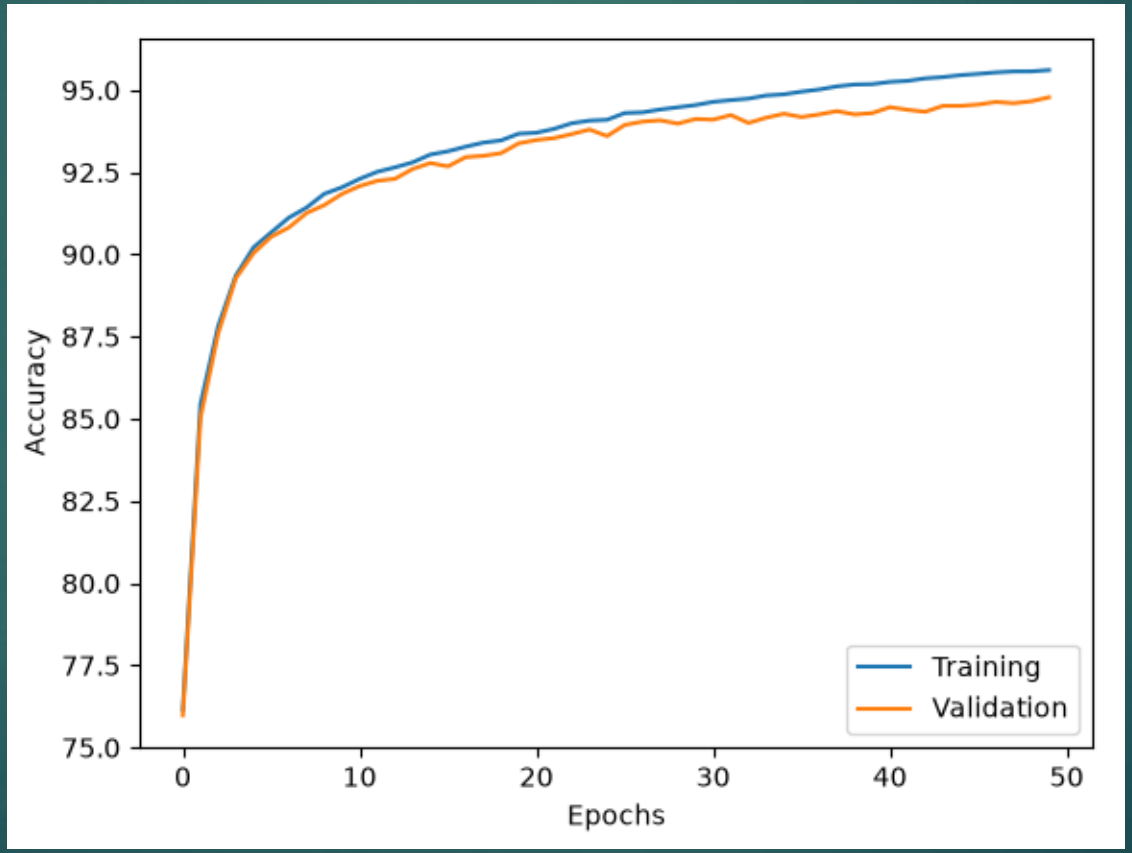
Training an MLP

- Loss curve



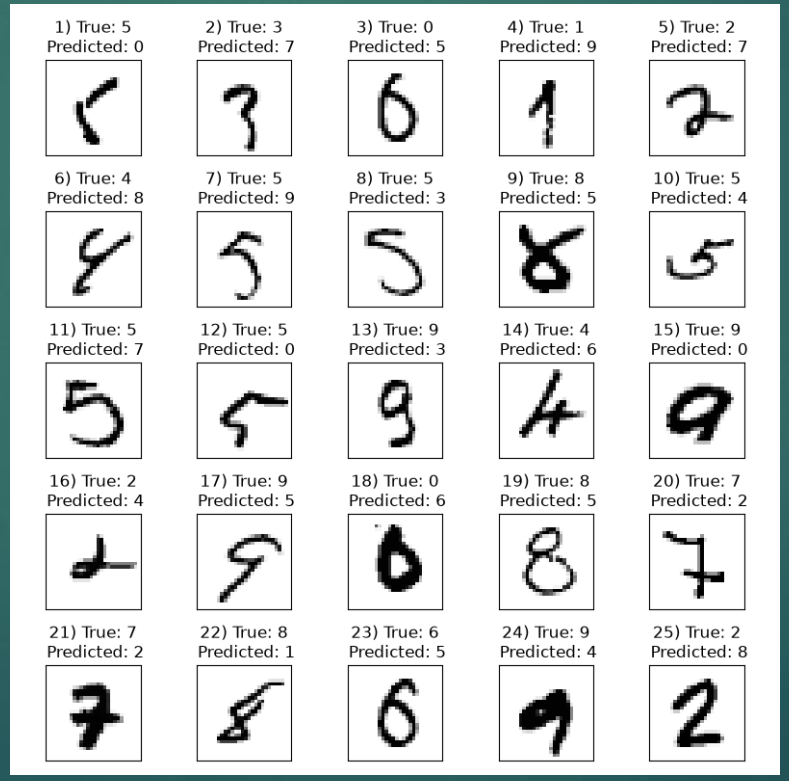
Training an MLP

- Training and validation accuracy



Evaluate the Performance

- Test accuracy `Test accuracy: 94.54%`
- The first 25 misclassified samples



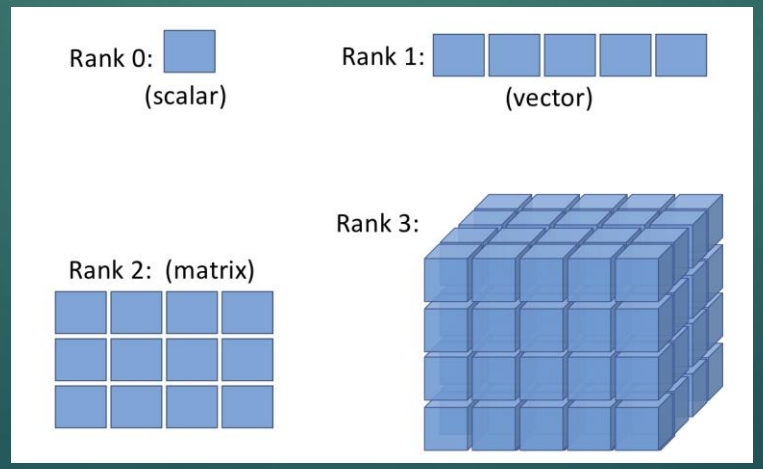
PyTorch

PyTorch

- PyTorch (torch) is one of the most popular deep learning libraries
  - Supports deep learning via a dynamic computation graph
    - Provides GPU acceleration, tensor computation, and automatic differentiation
    - Lets us implement NNs much more efficiently
  - Offers domain-specific libraries such as TorchText (torchtext), TorchVision (torchvision) and TorchAudio (torchaudio)
    - Provide a number of pre-loaded datasets
- Main features
  - Tensors, Autograd (automatic differentiation), nn.Module
- How to use? 🤔
  - Getting Started
  - User Guide / Tutorials
  - API Reference

Tensor (張量)

- Specialized data structure that are very similar to arrays and matrices
  - Used to encode the inputs and outputs of a model, as well as the model's parameters
  - Similar to NumPy's ndarrays, except that tensors are optimized for automatic differentiation and can run on GPUs



Tensor (張量)

- Creating a Tensor
  - From a list

```
data = [[1, 2],[3, 4]]
x_data = torch.tensor(data)
```

  - From a NumPy array

```
np_array = np.array(data)
x_np = torch.from_numpy(np_array)
```

  - With random or constant values

```
shape = (2,3)
rand_tensor = torch.rand(shape)
ones_tensor = torch.ones(shape)
zeros_tensor = torch.zeros(shape)
```

Tensor (張量)

- Attributes of a Tensor
  - Describe their shape, datatype, and the device on which they are stored

```
tensor = torch.rand(3,4)

print(f"Shape of tensor: {tensor.shape}")
print(f"Datatype of tensor: {tensor.dtype}")
print(f"Device tensor is stored on: {tensor.device}")
```

tensor.reshape()

```
Shape of tensor: torch.Size([3, 4])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu
```

```
# We move our tensor to the current accelerator if available
if torch.accelerator.is_available():
    tensor = tensor.to(torch.accelerator.current_accelerator())
```

By default, tensors are created on the CPU.
We need to *explicitly* move tensors to the accelerator (cuda or mps) using .to() method.

Tensor (張量)

- Operations on Tensors
  - Arithmetic
  - Linear algebra
    - Matrix manipulation
      - transposing, indexing, slicing, joining
  - Sampling
  - …

```
# This computes the matrix multiplication between two tensors. y1, y2, y3 will have the same value
# ``tensor.T`` returns the transpose of a tensor
y1 = tensor @ tensor.T
y2 = tensor.matmul(tensor.T)

y3 = torch.rand_like(y1)
torch.matmul(tensor, tensor.T, out=y3)


# This computes the element-wise product. z1, z2, z3 will have the same value
z1 = tensor * tensor
z2 = tensor.mul(tensor)

z3 = torch.rand_like(tensor)
torch.mul(tensor, tensor, out=z3)
```

```
tensor = torch.ones(4, 4)
print(f"First row: {tensor[0]}")
print(f"First column: {tensor[:, 0]}")
print(f"Last column: {tensor[..., -1]}")
tensor[:,1] = 0
```

torch.sum(), .mean(), .std(), .linalg.norm(), .transpose(), .squeeze(), .chunk(), .split(), .cat(), .stack()

Bridge with NumPy

- Tensors on the CPU and NumPy arrays can share their memory locations (changing one will change the other)
  - Tensor to NumPy array

```
t = torch.ones(5)
print(f"t: {t}")
n = t.numpy()
print(f"n: {n}")
```

```
t: tensor([1., 1., 1., 1., 1.])
n: [1. 1. 1. 1. 1.]
```

In-place add operation

```
t.add_(1)
print(f"t: {t}")
print(f"n: {n}")
```

```
t: tensor([2., 2., 2., 2., 2.])
n: [2. 2. 2. 2. 2.]
```

  - NumPy array to Tensor

```
n = np.ones(5)
t = torch.from_numpy(n)
```

```
np.add(n, 1, out=n)
print(f"t: {t}")
print(f"n: {n}")
```

```
t: tensor([2., 2., 2., 2., 2.], dtype=torch.float64)
n: [2. 2. 2. 2. 2.]
```

Operations that store the result into the operand are called *in-place*, denoted by a _ suffix.

General Modules

- Working with data: torch.utils.data
  - Dataset: Stores the samples and their corresponding labels
  - DataLoader: Wraps an iterable around the Dataset
- Building an NN model: torch.nn
  - Module
  - Sequential
  - torch.nn.functional
- Optimizer: torch.optim
- Computer vision: torchvision
  - datasets
  - transforms

PyTorch functions and classes often have parameters with default values that we *may* omit for clarity. You can read more about those parameters by going through the online documentation.

Dataset

```
>>> image_dataset = ImageDataset(file_list, labels, transform)
```

- Creating a *custom* Dataset (subclass) (自訂資料集)
  - Must implement three functions:
    - __init__: Run once when instantiating the Dataset object

```
class ImageDataset(Dataset):
    def __init__(self, file_list, labels, transform=None):
        self.file_list = file_list
        self.labels = labels
        self.transform = transform
```

    - __len__: Returns the number of samples in the dataset

```
def __len__(self):
    return len(self.labels)
```

    - __getitem__: Loads and returns a sample from the dataset at the given index

```
def __getitem__(self, index):
    img = Image.open(self.file_list[index])
    if self.transform is not None:
        img = self.transform(img)
    label = self.labels[index]
    return img, label
```

Dataset

- Creating an *image* Dataset
  - torchvision.datasets.ImageFolder is a generic data loader where images are arranged in *folders* by *class*

```
# Load dataset
dataset = datasets.ImageFolder(root='path/to/root', transform=transform)
```

```
root/dog/1.png
root/dog/2.png
root/cat/1.png
root/cat/2.png
```

Dataset

- Loading an *available* Dataset (載入內建資料集)
  - Example: Load the Fashion-MNIST dataset from TorchVision

```
import torch
from torch.utils.data import Dataset
from torchvision import datasets
from torchvision.transforms import v2
import matplotlib.pyplot as plt


training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])
)
```

- `root` is the path where the train/test data is stored,
- `train` specifies training or test dataset,
- `download=True` downloads the data from the internet if it's not available at `root`.
- `transform` and `target_transform` specify the feature and label transformations

DataLoader

- Wraps an iterable around the Dataset
  - Supports automatic batching, sampling, shuffling and multiprocess data loading

```
from torch.utils.data import DataLoader

train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=64, shuffle=True)
```

```
for X, y in test_dataloader:
    print(f"Shape of X [N, C, H, W]: {X.shape}")
    print(f"Shape of y: {y.shape} {y.dtype}")
    break
```

next(iter(test_dataloader))

enumerate(test_dataloader, start=1)

```
Shape of X [N, C, H, W]: torch.Size([64, 1, 28, 28])
Shape of y: torch.Size([64]) torch.int64
```

⋮

The Dataset retrieves our dataset's features and labels one sample at a time.

Transforms

- Perform manipulation of the data and make it suitable for training
  - transform: to modify the *features*
  - target_transform: to modify the *labels*

```python
import torch
import torch.nn.functional as F
from torchvision import datasets
from torchvision.transforms import v2

ds = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]),
    target_transform=v2.Lambda(
        lambda y: F.one_hot(torch.tensor(y), num_classes=10).float()
    ),
)
```

新版的 transforms

取代舊版的 transforms.ToTensor()

Build the Neural Network

- The torch.nn provides all the *building blocks* to build a neural network
  - A neural network is a module itself that consists of other modules (layers)
    - Every module in PyTorch subclasses the nn.Module which allows layers to be stacked to form a network
  - This *nested structure* allows for building and managing complex architectures easily
    - Each layer in an NN receives its inputs from the preceding layer; therefore, its dimensionality is fixed
    - Typically, we need to concern with the dimensionality of output only

Define the NN Model

- Subclassing nn.Module
  - Initialize the neural network layers in __init__
  - Implements the operations on input data in the forward method

```
class Model(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.layer2 = nn.Linear(hidden_size, output_size)
    def forward(self, x):
        x = self.layer1(x)
        x = nn.Sigmoid()(x)
        x = self.layer2(x)
        x = nn.Softmax(dim=1)(x)
        return x
```

≡ Fully connected layer (or dense layer)

若採用 nn.CrossEntropyLoss() 作為 Loss function，已經包含 nn.LogSoftmax 與 nn.NLLLoss，不應該在最後一層加上 nn.Softmax()

注意：網路的輸出層要與 Loss function 搭配

Create an Model Instance

```
input_size = X_train_norm.shape[1]
hidden_size = 16
output_size = 3
model = Model(input_size, hidden_size, output_size)
```

- To use the model, we pass it the input data

```
logits = model(X)
pred_probab = nn.Softmax(dim=1)(logits)
y_pred = pred_probab.argmax(1)
```

  - This executes the model's forward()
  - Do *not* call model.forward() directly

Model Parameters

- Subclassing nn.Module automatically tracks all fields defined inside the model object
  - Access parameters (e.g., weights and biases) using model's parameters() or named_parameters() methods

```
for name, param in model.named_parameters():
    print(f"Layer: {name} | Size: {param.size()} | Values : {param[:2]} \n")
```

(只印出前兩列)

```
Layer: linear_relu_stack.0.weight | Size: torch.Size([512, 784]) | Values : tensor([[-0.0138, -0.0303,  0.0029,  ...,  0.0055, -0.0071,  0.0274],
        [ 0.0009, -0.0344, -0.0046,  ..., -0.0242,  0.0308,  0.0048]],
       device='cuda:0', grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.0.bias | Size: torch.Size([512]) | Values : tensor([-0.0062,  0.0043], device='cuda:0', grad_fn=<SliceBackward0>)
```

Choose a Loss Function

- Mean Square Error (for regression)
  - nn.MSELoss
- *Binary* Cross Entropy (for binary classification or multilabel classification)
  - nn.BCELoss
    - Input: probabilities
  - 👍 nn.BCEWithLogitsLoss
    - Input: logits (不要經過 sigmoid)
    - Combines nn.Sigmoid and nn.BCELoss
- Negative Log Likelihood
  - nn.NLLLoss
    - Input: *log*-probabilities
- *Categorical* Cross Entropy
  - 👍 nn.CrossEntropyLoss
    - Input: logits (不要經過 softmax)
    - Combines nn.LogSoftmax and nn.NLLLoss

for multiclass classification

```
# Initialize the loss function
loss_fn = nn.CrossEntropyLoss()
```
(物件)

Categorical Cross Entropy (多分類交叉熵)

- Single sample loss

$$L = -\sum_{c=1}^{C} y_c \cdot \log(\hat{y}_c)$$

- C：總類別數量（例如 3 分類、10 分類）。
- y_c：真實標籤（True Label）中第 c 個類別的機率值。若使用 **One-hot 編碼**，正解類別為 `1`，其餘皆為 `0`。
- $\hat{y}_c$：模型預測該樣本屬於第 c 個類別的機率值（經過 **Softmax** 轉換後的值，介於 0 到 1 之間）。

  - 非正解類別的 y_c 都是 0，對應的乘法項次全部消失
    - 實質只專注計算「模型對正確答案所預測出的機率，取自然對數後再加負號」的數值
    - 當預測正確答案的機率越接近 1，對數值越接近 0，損失值就越低

Categorical Cross Entropy (多分類交叉熵)

- Batch average loss

$$Loss = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c} \cdot \log(\hat{y}_{i,c})$$

- $\boldsymbol{N}$：一個 Batch 中的樣本總數（Batch Size）。
- $\boldsymbol{y_{i,c}}$：第 i 個樣本在第 c 個類別的真實標籤。
- $\boldsymbol{\hat{y}_{i,c}}$：第 i 個樣本在第 c 個類別的預測機率。

Choose an Optimizer

- Optimization is the process of adjusting model parameters to reduce error in each training step
  - Optimization algorithms define how this process is performed
  - All optimization logic is encapsulated in the optimizer object
    - torch.optim.SGD, torch.optim.Adam, torch.optim.AdamW
- Initialize the optimizer by *registering* the model's parameters that *need to be trained*, and passing in the learning rate hyperparameter

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

Autograd

- Automatic differentiation with torch.autograd
  - Built-in differentiation engine to compute gradients
  - It supports *automatic* computation of gradient for any computational graph
    - A function that we apply to tensors to construct computational graph is a subclass of torch.autograd.Function
      - Computes the function in the *forward* direction
      - Computes its derivative during the *backward* propagation step

```
import torch

x = torch.ones(5)  # input tensor
y = torch.zeros(3)  # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w)+b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
```

Declare Tensors for which gradients should be computed with requires_grad=True

(函式)

Optimization Powered by Autograd

- Inside the training loop, optimization happens in three steps: (the magic of Autograd)
  - Call loss.backward() to *compute gradients* w.r.t. each parameter via backpropagating the prediction loss
  - Call optimizer.step() to *update parameters* by the gradients
  - Call optimizer.zero_grad() to *reset the gradients* of parameters
    - Gradients by default add up
    - Explicitly zero them at each iteration to prevent double-counting

Training the Model

- Train and optimize the model with an optimization loop
  - Each epoch consists of two main parts
    - The train loop
    - The validation/test loop

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

epochs = 10
for t in range(epochs):
    print(f"Epoch {t+1}\n-------------------------------")
    train_loop(train_dataloader, model, loss_fn, optimizer)
    test_loop(test_dataloader, model, loss_fn)
print("Done!")
```

Train Loop

```
def train_loop(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    # Set the model to training mode - important for batch normalization and dropout layers
    # Unnecessary in this situation but added for best practices
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        # Compute prediction and loss
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        if batch % 100 == 0:
            loss, current = loss.item(), batch * batch_size + len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")
```

1. Generate predictions
2. Calculate loss
3. Compute gradients
4. Update parameters using gradients
5. Reset the gradients to zero

Validation/Test Loop

```python
def test_loop(dataloader, model, loss_fn):
    # Set the model to evaluation mode - important for batch normalization and dropout layers
    # Unnecessary in this situation but added for best practices
    model.eval()
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    test_loss, correct = 0, 0

    # Evaluating the model with torch.no_grad() ensures that no gradients are computed during test mode
    # also serves to reduce unnecessary gradient computations and memory usage for tensors with requires_grad=True
    with torch.no_grad():
        for X, y in dataloader:
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()

    test_loss /= num_batches
    correct /= size
    print(f"Test Error: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f} \n")
```

Make predictions only

Save and Load the Whole Model

- Save both the model architecture and all the learned parameters
  - Save

Using a 'pt' or 'pth' file extension just for convention

```
path = 'iris_classifier.pt'
torch.save(model, path)
```

Use Python pickle module to serialize the model

  - Load

```
model_new = torch.load(path)
model_new.eval()
```

Be sure to call model.eval() method before inferencing.

Save and Load the Model Weights Only

- Save only the learned parameters
  - Save
    - Learned parameters stored in an *internal dictionary* state_dict

```
path = 'iris_classifier_state.pt'
torch.save(model.state_dict(), path)
```

  - Load
    - First need to create an instance of the *same* model
    - Then load the parameters using load_state_dict() method

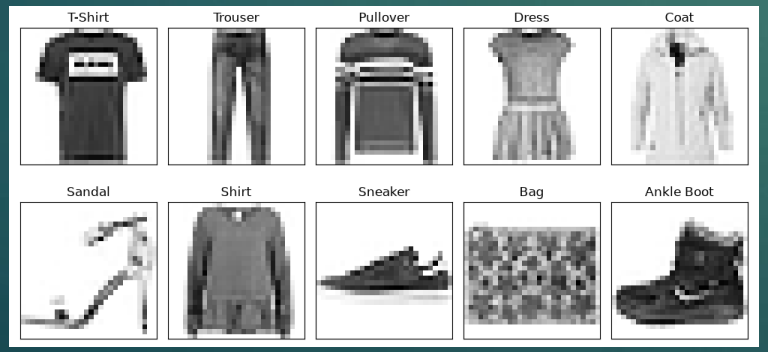
```
model_new = Model(input_size, hidden_size, output_size)
model_new.load_state_dict(torch.load(path))
```

Be sure to call model.eval() method before inferencing.

Building and Training an MLP in PyTorch

Classifying Fashion Products

- Build and train an MLP to classify fashion products from the FashionMNIST dataset
  - A MNIST-like database
  - 70,000 images of resolution 28×28 (grayscale)
  - Features: 784 pixels (integer $[0, 255]$)
  - Labels: (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) (10 classes)





Label	Description
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

T恤
褲子
套頭衫
洋裝
外套
涼鞋
襯衫
運動鞋
包包
短靴

Device and Hyperparameters Settings

- Sample code: 07_2.py

```python
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import v2

# 1. 自動偵測可用硬體加速 (CUDA / MPS / CPU)
device = torch.device(
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)
print(f"Using {device} device")
```

```python
# Hyperparameters
learning_rate = 1e-3
batch_size = 64
epochs = 20
```

Load Dataset and Define DataLoader

```python
# 2. 取得資料集
training_data = datasets.FashionMNIST(
    root="FashionMNIST",
    train=True,
    download=True,
    transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]),
)

test_data = datasets.FashionMNIST(
    root="FashionMNIST",
    train=False,
    download=True,
    transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]),
)

# 3. 創建 DataLoader
train_dataloader = DataLoader(
    training_data, batch_size=batch_size, shuffle=True
)
test_dataloader = DataLoader(test_data, batch_size=batch_size, shuffle=False)
```

```python
# Fashion-MNIST 類別標籤文字
labels_map = {
    0: "T-Shirt",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle Boot",
}
```

Batch shape:
$(64, 1, 28, 28)$
(N, C, H, W)

Build the MLP (Two Hidden Layers)

```
61  # 4. 定義 NN Model
62  class NeuralNetwork(nn.Module):
63
64      def __init__(self):
65          super().__init__()
66          self.flatten = nn.Flatten()
67          self.linear_relu_stack = nn.Sequential(
68              nn.Linear(28 * 28, 512),
69              nn.ReLU(),
70              nn.Linear(512, 512),
71              nn.ReLU(),
72              nn.Linear(512, 10),
73          )
74
75      def forward(self, x):
76          x = self.flatten(x)
77          logits = self.linear_relu_stack(x)
78          return logits
79
80  model = NeuralNetwork().to(device)  # 將模型移至指定硬體
```

Input layer (784 features)

Hidden layer 1 (512 nodes)

Hidden layer 2 (512 nodes)

Output layer (10 nodes)

哇~ 深度神經網路耶~ 😎

Without softmax

Define Train Loop

```
82  # 5. 定義 Train Loop
83  def train_loop(dataloader, model, loss_fn, optimizer):
84      model.train()
85      total_loss = 0
86      for batch, (X, y) in enumerate(dataloader):
87          X, y = X.to(device), y.to(device)  # 資料移至指定硬體
88
89          pred = model(X)
90          loss = loss_fn(pred, y)
91          loss.backward()
92          optimizer.step()
93          optimizer.zero_grad()
94
95          total_loss += loss.item()
96
97      return total_loss / len(dataloader)
```

Define Test Loop

```python
# 6. 定義 Test Loop
def test_loop(dataloader, model, loss_fn):
    model.eval()
    num_batches = len(dataloader)
    test_loss = 0

    all_preds = []
    all_trues = []
    all_images = []

    with torch.no_grad():
        for X, y in dataloader:
            X_dev, y_dev = X.to(device), y.to(device)
            pred = model(X_dev)

            test_loss += loss_fn(pred, y_dev).item()

            # 將結果移回 CPU 並轉為 NumPy 方便後續統計與繪圖
            preds = pred.argmax(1).cpu().numpy()
            trues = y.numpy()

            all_preds.extend(preds)
            all_trues.extend(trues)
            all_images.extend(X.numpy())

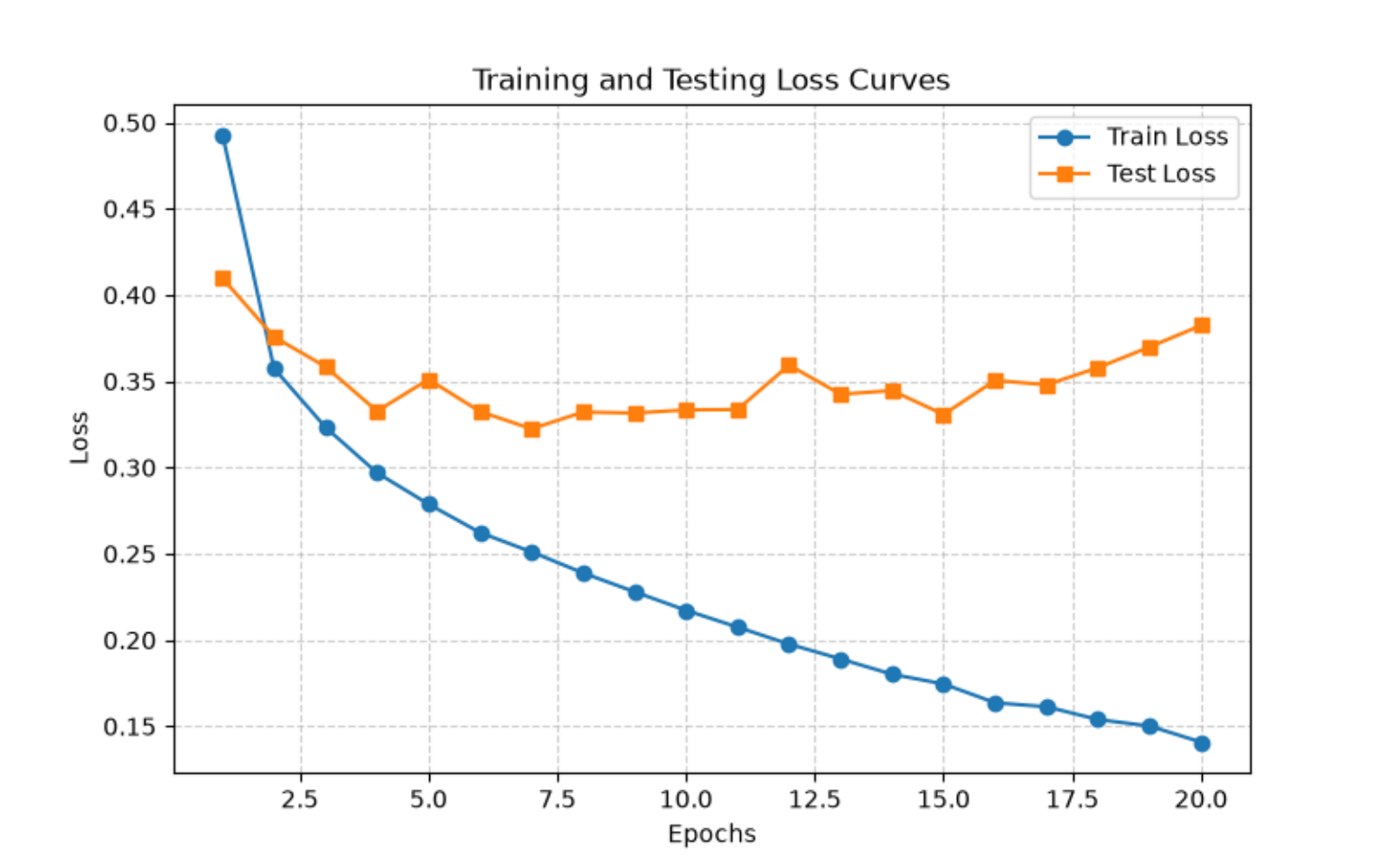
    avg_loss = test_loss / num_batches
    return avg_loss, np.array(all_preds), np.array(all_trues), np.array(all_images)
```

Choose Loss function and Optimizer

```
127  # 7. 選擇 Loss function 與 Optimizer
128  loss_fn = nn.CrossEntropyLoss()
129  optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
130
131  # 8. 訓練與測試模型
132  train_loss_history = []
133  test_loss_history = []
134
135  print("Start Training...")
136  for t in range(epochs):
137      train_loss = train_loop(train_dataloader, model, loss_fn, optimizer)
138      test_loss, preds, trues, images = test_loop(test_dataloader, model, loss_fn)
139
140      train_loss_history.append(train_loss)
141      test_loss_history.append(test_loss)
142
143      print(f"Epoch {t+1:02d} | Train Loss: {train_loss:.4f} | Test Loss: {test_loss:.4f}")
144  print("Done!\n")
```

Categorical cross entropy

Loss Curves


Training and Testing Loss Curves
Train Loss
Test Loss
Loss
0.50
0.45
0.40
0.35
0.30
0.25
0.20
0.15
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epochs

Classification Report

- Sample code: 07_3.py

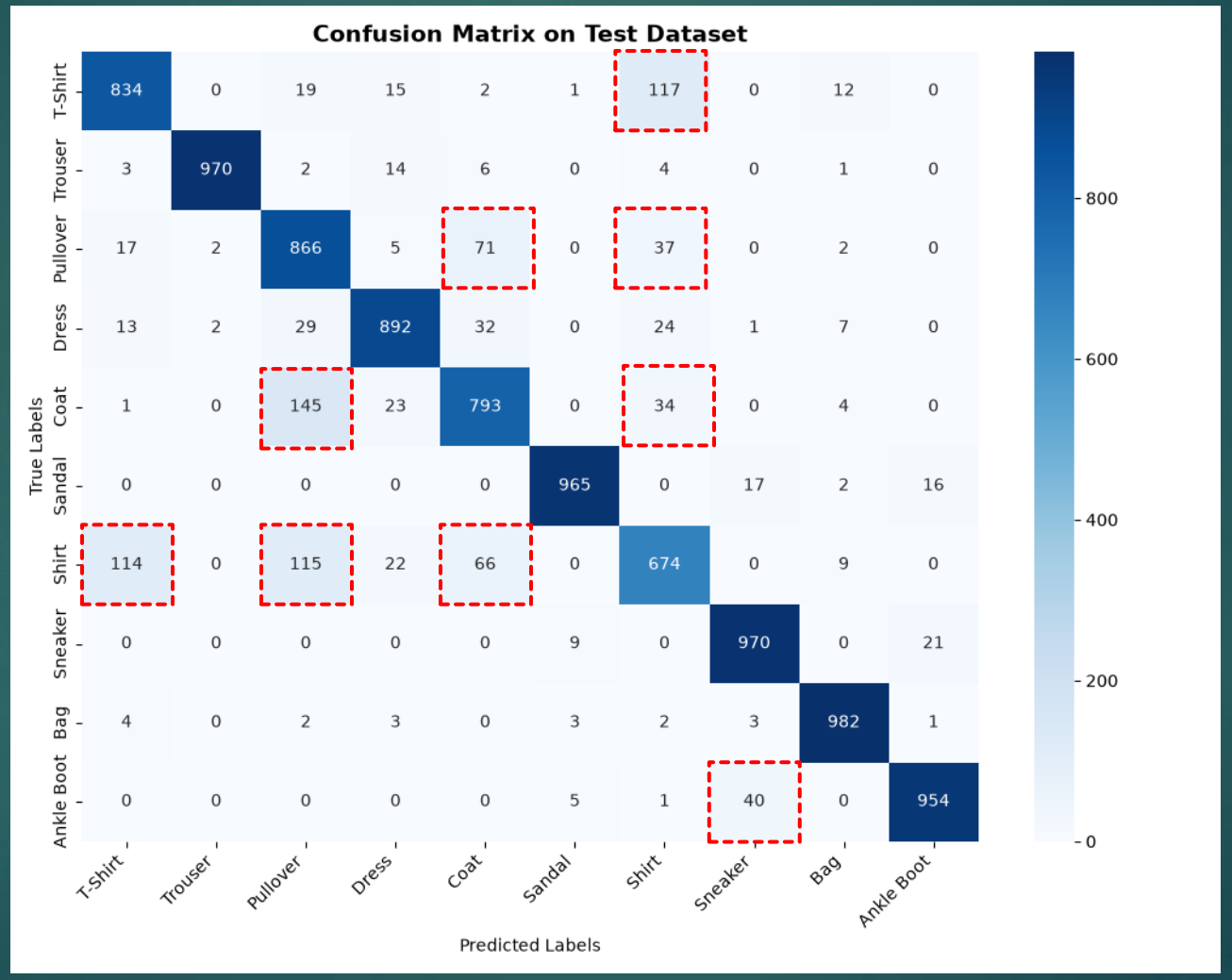
```
詳細分類報告 (Classification Report):
              precision    recall  f1-score   support

     T-Shirt       0.85      0.83      0.84      1000
     Trouser       1.00      0.97      0.98      1000
    Pullover       0.74      0.87      0.80      1000
       Dress       0.92      0.89      0.90      1000
        Coat       0.82      0.79      0.81      1000
      Sandal       0.98      0.96      0.97      1000
       Shirt       0.75      0.67      0.71      1000
     Sneaker       0.94      0.97      0.96      1000
         Bag       0.96      0.98      0.97      1000
  Ankle Boot       0.96      0.95      0.96      1000

    accuracy                           0.89     10000
   macro avg       0.89      0.89      0.89     10000
weighted avg       0.89      0.89      0.89     10000

Total misclassified samples: 1100 / 10000
```

Confusion Matrix



25 Misclassified Samples



Thank you